Ontology Development Report

Team AgenticCapabilities (use case AC-001) | Ontology Engineering 2026, Assignment 6 | October 16, 2026

Team members: Gia Oriana Santos, Barbara Kotlan, Jiaying (Michael) Wang, Jialin (Gary) Fang, Elioenai Lincon Guedes Alves

1. Overview

We built OE_6_TeamAgenticCapabilities.rdf from the Assignment 5 conceptual model. The work was split by figure of that model: Gia took Figures 1 and 3, Barbara Figure 2, Gary Figures 4 and 7, Jiaying Figures 5 and 8, and Lincon Figure 6. Gia merged the parts into one file. The ontology reuses AgentO, P-Plan, OWL-S Process, PROV-O, Commons and BEAM, and defines 65 classes, 33 object properties, 9 datatype properties and 11 individuals of its own. Pellet and HermiT find it consistent with no unsatisfiable classes, and with example individuals it answers our three focus competency questions, CQ 1, CQ 3 and CQ 5. The revised use case, term list and conceptual model were aligned with the merged file. Each member's part below covers what they found difficult, what worked, and how they used generative AI.

2. Individual contributions

Gia Oriana Santos

One of the main challenges I came across multiple times while working on the ontology is determining whether or not I should reuse certain classes and properties from another ontology. For our agentic capabilities use case, we were reusing ontologies from AgentO, P-Plan, PROV-O, OWL-S, Commons, and BEAM. Before starting the ontology, we already planned on reusing these ontologies and mapped them out on our conceptual model. As I was working on my part of the ontology while following the conceptual model, I realized that some of the concepts we were reusing didn't match our desired definitions for those concepts. For example, we initially planned to reuse agentO:WorkflowStep, but it was defined as "An individual action or phase within a workflow pattern to achieve the team goals", even though our use case doesn't address a workflow pattern or team goal like in the original AgentO ontology. For that reason, I decided to use ChatGPT Desktop Auto to help me make ontology design decisions. I mainly used the model to ask whether I should make a new concept in our ontology or make it a subclass of an existing concept in another ontology for reuse. I also used the model to define concepts from another ontology, and that also helped me make ontology decisions. The model helped me make decisions such as making an OrchestrationWorkflowStep class but making it a subclass of agentO:WorkflowStep so we can inherit agento:hasAssociatedTask, agentO:StartStep, and agentO:EndStep, while being able to define other new properties of OrchestrationWorkflowStep in our own terms. The rest of the difficult modeling decisions I had to make were choosing to reuse the same class from another ontology, making our own class inherit from another class from an existing ontology, or deciding not to reuse a class/relationship from another ontology at all.

Barbara Kotlan

I built Figure 2 of the conceptual model and its part of the ontology: requests, the information requirements and goals they express, and the constraints that steer plan selection. The hard part was deciding what a constraint is. A hard constraint must hold and a plan that breaks it is rejected; a soft constraint is only something to optimize, with a priority the user gives it, so it never gets "violated". Working that out also settled what a conflict is: two soft constraints conflict only when they push the same criterion in opposite directions, like maximizing and minimizing tool use, while constraints that just trade off, like strong evidence against few tool calls, are not conflicts, their priorities weight them. What worked was recording where each constraint came from with prov:wasDerivedFrom, so a constraint is traceable to the request or to a saved preference setting, which is how CQ 3 finds Jessica's safety-screening constraint. I also revised the term list after Danielle's A5 comments: identifiers became IRIs, every preferred label got a source with a link, the notes on how definitions were generated moved into a new editorial note column, and labels and definitions were updated to match the ontology.

Generative AI. I used ChatGPT Desktop to look up sources and links for the preferred labels and to check definitions against the ontology, giving it the term, its definition and the source we already had, and asking for a citable source for the label. That worked for most terms, and I kept those sources. It did not work for our own coined terms, where it only pointed back at our model, and some of its working notes ended up in the editorial column and had to be cleaned out afterwards.

Jiaying (Michael) Wang

I built Figures 5 and 8 of the conceptual model and their part of the ontology: orchestration plans, their steps and task dependencies, the kinds and roles of information artifacts, and the evidence model. After the merge I checked the ontology against the course rules and the A5 feedback, fixed labels, definitions and names, redrew the conceptual model as Version 2.0 so that every class and arrow in it is a term of the ontology, and aligned the term list with the ontology. The hardest part was the merge. Five of us had modeled overlapping terms (my hasWorkflowStep against Gia's hasStep, my Parameter under OWL-S against the team's own Parameter), and a disjointness axiom over 45 imported classes made 26 classes unsatisfiable, AgentO's LLMAgent among them. Protégé showed nothing wrong until the reasoner ran. What worked was deriving relations instead of entering them: dependsOn is consumes ∘ produces^{-1} and hasTask is hasStep ∘ hasAssociatedTask, so an example plan only states what each task consumes and produces, and Pellet derives which steps can run in parallel and, for CQ 5, the capability behind each task. A script that compares every diagram element with the ontology caught names that had drifted apart (satisfies, violates, MCPTool) before the submission.

Generative AI. I used Claude Opus 5.5 in Claude Code. It was given the A6 handout, the A5 feedback, the merged ontology, the A5 conceptual-model sources, the A5 use case and the team's term list. The prompt only set the goal: find what could still cost points, and mark what the team had to decide. Claude wrote scripts that helps me apply each fix to the ontology, re-serialize it with the course serializer, run Pellet and HermiT. And also check every diagram element against the ontology.

Jialin (Gary) Fang

I built Figures 4 and 7 of the conceptual model and the matching part of the ontology: tools, the servers that expose them and their parameters, and the invocations with their provenance. Most of it reuses OWL-S and PROV-O, so the hard part was not finding terms but attaching them to our own classes without wrong inferences. For example, Tool became a subclass of both agento:Tool and process:Process so the reused properties apply to it directly, but a skill's precondition needed our own hasPrecondition, because the OWL-S version would have made every skill a process. What worked well was treating a tool call as a class: a ToolInvocation is part of the SkillInvocation that made it and points back to its planned step, which is exactly what CQ 4 and CQ 5 need to explain a plan. I also fixed the term list after Danielle's A5 comments (one definition source per term, no "AC-001" or "project terminology" as a source, real sources for our own terms) and added the 35 terms that Version 2.0 of the model introduced.

Generative AI. I used Claude Fable 5.1 in Claude Code on 15 and 16 October. I gave it our team folder, the A6 handout, Danielle's comments, Barbara's revised list and the Version 2.0 model, and asked in plain words to check whether the term list fixed the TA comments and to make the changes it found. It worked well on the mechanical part: it compared every version of the list cell by cell, edited the spreadsheet without breaking the comments or formatting, and highlighted and logged every change. I kept its fixes and the sources it looked up and quoted.

Elioenai Lincon Guedes Alves

I modeled Figure 6 of the conceptual model, plan evaluation and selection, which Gia added to the ontology on 10/12. The hard part was the open-world assumption: the knowledge graph cannot conclude that a plan was rejected

just because it is not marked as selected, so the harness has to state both, and selected and rejected plans are disjoint classes. The other decision was that only hard constraints can be violated. A candidate plan that violates one is classified as rejected by a general class axiom, so the reasoner does the first filtering step; the remaining candidates are ranked by their weighted criterion scores, and soft constraints only lower a plan's score and rank. What worked was the n-ary pattern for a plan evaluation: one evaluation record holds the candidate plan, the criteria used, the metrics measured, and the score and rank, which is enough to answer CQ 1 for Ken (plan 1 scores 2.00 and ranks first, plan 2 scores 0.00) and CQ 6 for Olivia.

Generative AI. I used Claude Opus 5.5 while drawing Figure 6. My main prompt was here is our plan-evaluation figure and the W3C n-ary relations note. Check whether a reasoner could classify a plan as rejected from a violated hard constraint, and show the axiom in Manchester syntax.". What I kept was the general class axiom that classifies a candidate plan as rejected when it violates a hard constraint; what I dropped was its suggestion to use a single "violates" property with a severity value, because the reasoner cannot classify plans on it.

3. Decisions that may be controversial

- **Only hard constraints are violated.** violatesConstraint links a candidate plan only to a HardConstraint, and an axiom classifies a candidate that violates one as a RejectedOrchestrationPlan. A soft constraint only lowers a plan's score and rank. This answers the A5 comment on hard versus soft constraints. A single "violates" with a severity value was rejected because the reasoner cannot classify plans on it.
- **Our own step class under AgentO's WorkflowStep.** AgentO defines a workflow step inside a team workflow pattern, which our use case does not have, so we did not reuse it directly. OrchestrationWorkflowStep is a subclass of it: it inherits agento:hasAssociatedTask and sits beside AgentO's StartStep and EndStep, and the team defines its own properties for it (see Gia's part).
- **No disjointness across imported vocabularies.** Disjointness is stated only between sibling classes the team owns: selected and rejected plans, the kinds of skill resource and of plan metric, and user and execution constraints. Declaring 45 imported classes pairwise disjoint contradicted AgentO, where an LLMAgent is both a prov:Agent and an agento:Tool, and made 26 classes unsatisfiable.
- **Reuse as published, or a team term.** AgentO, P-Plan, OWL-S and PROV-O terms are reused unchanged wherever their domain and range fit. The team defines its own term where reuse would force a wrong inference: a skill's precondition uses hasPrecondition because OWL-S's domain would make every skill a process, and a skill performing a task uses performedBySkill because agento:performedBy's range would make every skill a tool.
- **Derived relations.** dependsOn and hasTask are property chains, so a stated dependency can never disagree with the artifacts and steps. The cost is that they cannot be used in cardinality restrictions.
- **Course naming rules over the source's names.** Datatype properties became verbs (score → hasScore and six more), MCPTool became ModelContextProtocolTool, and every label is lower case, as Ms. Kendall's review asked. The term list and the diagrams use the same names.